## EJERCICIO 1

Utilizando un conjunto de Pascal, añade una operación al TAD `ListaEnteros` que permita eliminar todos los valores duplicados de una lista. *Complejidad de tiempo esperada: O(n). Espacio auxiliar esperado: O(n).*

**SOLUCIÓN:**

```
procedure remove_duplicates(var list:tlistaEnteros);
var
  conjunto_aux: set of 0..255;
  aux_list: tlistaEnteros;
  x: integer;
begin
  initialize(aux_list);
  conjunto_aux:= [];  // inicializamos el conjunto
  while not is_empty(list) do
  begin
       x:= first(list);
       delete_at_begin(list);
       if not (x in conjunto_aux) then
         begin
            include(conjunto_aux,x);
            insert_at_end(aux_list,x);
         end;
  end;
  list:=aux_list;
end;
```

**Nota**: Las limitaciones de los sets en Pascal hacen que solo podamos eliminar de la lista valores duplicados que se encuentren entre 0 y 255.

## EJERCICIO 1bis

Repite el ejercicio anterior aplicando esta vez la implementación al TAD ListaGenérica y comenta qué modificaciones serían necesarias. *Complejidad de tiempo esperada: O(n). Espacio auxiliar esperado: O(n).*

**SOLUCIÓN:** *(las modificaciones van resaltadas en amarillo)*

```
procedure remove_duplicates(var list: tlistaGenerica);
var
  conjunto_aux: set of 0..255; // Asumimos las limitaciones de los sets en Pascal
  aux_list: tlistaGenerica;
  x: TElemento;
  x_value: integer;
begin
  initialize(aux_list);
  conjunto_aux:=[];
  while not is_empty(list) do
  begin
       assign(x,first(list));
       x_value := value(x); // extrae el valor entero para el conjunto
       delete_at_begin(list);
       if not (x_value in conjunto_aux) then
         begin
            include(conjunto_aux, x_value);
            insert_at_end(aux_list,x);
         end;
  end;
  list:=aux_list;
end;
```

## EJERCICIO 2

Implementa un TAD `ConjuntoStrings` donde el tipo base serán cadenas de texto de cualquier longitud. Programa las operaciones básicas esenciales del TAD en una Unit de Pascal utilizando una lista ordenada como representación interna.

**SOLUCIÓN:**

*Asumiendo que tenemos implementado un TAD uSortedListOfStrings…*

```
unit uSortedListOfStrings;

interface
  type
    nodo = record
      info: string;
      next: ^nodo;
    end;

    tSortedListOfStrings = record
      first: ^nodo
    end;

  { Operaciones básicas }
  procedure initialize(var list: tSortedListOfStrings);
  function is_empty(list: tSortedListOfStrings): boolean;
  function in_list(list: tSortedListOfStrings; s:string): boolean;
  procedure insert(var list: tSortedListOfStrings; s: string);
  procedure delete(var list: tSortedListOfStrings; s: string);

  { Otras operaciones }
  procedure show(list: tSortedListOfStrings);
  function pop(var list: tSortedListOfStrings): string; //saca el primero de la lista
```

*…el código del conjunto sería:*

```
unit uConjuntoStrings;

interface

   uses
      usortedListOfStrings;

   type
        tConjuntoStrings = record
          contenedor: tSortedListOfStrings;
          size: integer;
        end;

   procedure initialize(var c: tConjuntoStrings);
   procedure add(var c: tConjuntoStrings; s: string);
   procedure remove(var c: tConjuntoStrings; s: string);
   function in_set(c: tConjuntoStrings; s: string): boolean;
   function size(c: tConjuntoStrings): integer;
   procedure show(c: tConjuntoStrings);

implementation

   procedure initialize(var c: tConjuntoStrings);
   begin
     usortedListOfStrings.initialize(c.contenedor);
     c.size:= 0;
```

```
  end;

  function size(c: tConjuntoStrings): integer;
  begin
    size:= c.size;
  end;

  procedure add(var c: tConjuntoStrings; s: string);
  begin
    if not in_set(c,s) then
    begin
       insert(c.contenedor, s);
       c.size:= c.size + 1;
    end;
  end;

  procedure remove(var c: tConjuntoStrings; s: string);
  begin
    if in_set(c,s) then
    begin
       c.size:= c.size - 1;
       delete(c.contenedor,s);
    end;
  end;

  function in_set(c: tConjuntoStrings; s: string): boolean;
  begin
     in_set:= in_list(c.contenedor,s);
  end;

  procedure show(c: tConjuntoStrings);
  begin
    usortedListOfStrings.show(c.contenedor);
  end;

end.
```

## EJERCICIO 3

Implementa las operaciones usuales entre conjuntos (Unión, Intersección, Diferencia e Inclusión) sobre el TAD `ConjuntoStrings` programado en el ejercicio 2.

```
  function union(var c1,c2: tConjuntoStrings): tConjuntoStrings;
  var
    union_set: tConjuntoStrings;
    aux_container: tSortedListOfStrings;
    s: string;
  begin
    initialize(union_set);
    uSortedListOfStrings.initialize(aux_container);
    while not (is_empty(c1.container)) do
    begin
       s:= pop(c1.container);
       add(union_set,s);
       insert(aux_container,s);
    end;
    c1.container:= aux_container;
    uSortedListOfStrings.initialize(aux_container);
    while not (is_empty(c2.container)) do
    begin
       s:= pop(c2.container);
       add(union_set,s);
       insert(aux_container,s);
    end;
```

```
  c2.container:= aux_container;
  union:= union_set;
end;

function intersection(var c1,c2: tConjuntoStrings): tConjuntoStrings;
var
  intersection_set: tConjuntoStrings;
  aux_container: tSortedListOfStrings;
  s: string;
begin
  initialize(intersection_set);
  uSortedListOfStrings.initialize(aux_container);
  while not is_empty(c1.container) do
  begin
     s:= pop(c1.container);
     insert(aux_container,s);
     if in_set(c2,s) then add(intersection_set,s);
  end;
  c1.container:= aux_container;
  intersection:= intersection_set;
end;

function difference(c1,c2: tConjuntoStrings): tConjuntoStrings;
var
  diff_set: tConjuntoStrings;
  aux_container: tSortedListOfStrings;
  s: string;
begin
  initialize(diff_set);
  uSortedListOfStrings.initialize(aux_container);
  while not is_empty(c1.container) do
  begin
     s:= pop(c1.container);
     insert(aux_container,s);
     if not in_set(c2,s) then add(diff_set,s);
  end;
  c1.container:= aux_container;
  difference:= diff_set;
end;

function included_in(var c1,c2: tConjuntoStrings): boolean;
var
  aux_container: tSortedListOfStrings;
  s: string;
begin
  included_in:= true;
  uSortedListOfStrings.initialize(aux_container);
  while (not is_empty(c1.container)) and (included_in) do
  begin
     s:= pop(c1.container);
     insert(aux_container,s);
     if not in_set(c2,s) then included_in:= false;
  end;
  while not is_empty(aux_container) do begin
     s:= pop(aux_container);
     insert(c1.container,s);
  end;
end;
```

## EJERCICIO 4 (reto)

Implementa el TAD `BolsaStrings` similar a los TADs programados en los ejercicios anteriores, pero utilizando como representación interna una tabla hash. Deberás programar también la función hash, para lo cual deberás investigar un poco al respecto.

## EJERCICIO 5 (largo)

Queremos implementar un juego de bingo en Pascal. El bingo consta de un bombo de 90 bolas (numeradas del 1 al 90) que se van extrayendo y de tantos cartones como jugadores con 10 números del 1 al 90 en cada cartón. Se pide:

a) Implementar el TAD `Bombo_Bingo` utilizando un conjunto de enteros del 1 al 90 cuyas operaciones básicas serán:

- `inicializar`: deja el bombo listo para empezar a jugar
- `extraer_numero`: elige un número de los que aún no han salido y lo retorna
- `introducir_numero`: es algo raro, pero a veces necesario devolver un número al bombo, lo cual se hace con esta operación
- `restantes`: permite consulta cuántos números quedan en el bombo.

Para la implementación interna del conjunto, utiliza un array de booleanos.

b) Implementar el TAD `Cartón_Bingo`, que tendrá 15 enteros no repetidos entre 1 y 90 y que incluirá las siguientes operaciones:

- `inicializar`: genera 15 números aleatorios y los deja listos para empezar a jugar
- `tachar_numero`: cuando sale del bombo un número que está en su cartón, el jugador debe marcarlo utilizando este método
- `consultar_numero`: permite consultar si un cierto número está en el cartón
- `consultar_linea`: Se dice que un cartón “canta línea” cuando tiene 5 números de una fila consecutivos ya marcados porque han salido en el juego. Asumiremos que la primera línea son los 5 primeros números, la segunda los 5 siguientes y la tercera los 5 últimos.
- `consultar_bingo`: Se dice que un cartón “canta bingo” cuando tiene los 15 números ya marcados porque han salido en el juego.

c) Implementar el juego siguiendo el siguiente proceso simplificado:

- Inicializar el bombo y tras determinar el número de cartones, inicializarlos
- Ir sacando sucesivamente números y anotándolos en los cartones.
- Cada vez que sale un número, se debe comprobar si hay línea o bingo en algún cartón y si lo hay notificarlo (sacar un msg por pantalla).
- Una vez cantada línea por parte de algún cartón, nadie más puede cantar línea.
- El juego termina cuando algún cartón canta bingo.